VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Aashwit Suhane	24BSA10182
Slot	**Date of Submission**
E11	31-03-2026

1. Introduction

For this report I chose Git. I didn’t pick it randomly because I had already used it a little before, mainly for uploading code and cloning repositories. I also used git push while working on my DSA problems, although at that time I was mostly just following steps and not really thinking about what was happening internally. So I thought it would make more sense to pick something I had at least seen before.

Git is basically a version control system, but honestly that definition didn’t help me much in the beginning. Earlier I thought it is just for storing code somewhere online, but it’s actually more about tracking changes. Like if something breaks, we can go back to an older version. I didn’t really use this feature properly before, but now it makes more sense why it is important.

One thing that confused me a lot was Git vs GitHub. I used to think both are the same thing. Later I understood that Git is the actual tool and GitHub is just a platform built on top of it. Git can work without GitHub, which I didn’t know earlier, so that cleared up some confusion.

While doing this report, I tried to look beyond just commands like git init or git clone. I wanted to understand why Git was created and how it fits into open-source software. From what I understood, it was made to manage large codebases and help multiple developers work together without too much conflict.

I also explored how Git works on a Linux system. It felt a bit different from other software because it doesn’t run in the background as a service. It only works when we actually run commands. Another thing I noticed was the .git folder inside a project directory. I had seen it earlier but never opened it properly. This time I checked it and saw many files, although I didn’t fully understand everything inside.

I also looked at how Git connects with other tools. It is used with platforms like GitHub and GitLab, and also in web development. So even though Git itself is just a tool, it is part of a bigger system.

There are still some parts I am not completely sure about, especially the deeper working of Git, but overall this helped me understand it better than before. Earlier I was just using commands without thinking much, now I have at least some idea of what is happening.

So overall, this report is my attempt to understand Git not just as a tool, but as an example of how open-source software actually works in real situations..

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: ~~***Git***~~

3. Project Report Structure

Your report should be 12 to 16 pages long. Each section below maps to a unit in the course. Do not pad with screenshots — every page should contain original thinking and analysis written in your own words.

Part A — Origin and Philosophy (Units 1 & 2)

This is the most important section of the report. Allocate at least four pages to it.

A1. The problem this software was created to solve

Before Git was created, handling big software projects was not easy. The Linux kernel had many developers working from different places, so managing code changes was becoming confusing.

At that time they were using BitKeeper. It was useful, but it was not open source. So even though Linux itself is open, they were depending on a tool controlled by a company.

After some time, there was a problem between the Linux developers and the company. Because of that, they lost access to BitKeeper. This created a serious issue because suddenly there was no proper system to manage such a large project.

Then Linus Torvalds decided to make a new system himself. He wanted something fast and simple, and also something that does not depend on any central control. So developers could work more freely.

This is how Git started. It works in a distributed way, so every developer has their own copy of the project. This makes it more flexible and also safer.

Git was also made open source, so anyone can use it and improve it. Over time it became very popular and now it is used in most software projects.

What is your software's origin story? Tell it properly.

A2. The license — what it actually says

Git uses the GPL version 2 license, which is a type of open source license. It allows users to use, study, modify, and share the software, but it also has some strict conditions.

The four freedoms of free software are basically about what a user is allowed to do. First is the freedom to run the program for any purpose. Second is the freedom to study how the program works, which means access to source code is needed. Third is the freedom to modify the program. And fourth is the freedom to share copies with others. Git’s license gives all these freedoms, so in that sense it is fully open source.

About companies using it, a company can take Git and modify it, but if they distribute that modified version, they have to share their changes also. They cannot keep it closed. This is because GPL license follows a rule that modified versions must also remain open. So they can sell it, but they must also provide the source code.

There is a difference between GPL and MIT license. GPL is more strict, while MIT is more relaxed. In MIT license, a company can take the code, modify it, and even sell it without sharing the changes. But in GPL, sharing the changes is required if the software is distributed. So GPL tries to keep software open, while MIT gives more freedom to companies.

If I had to choose, I would probably choose MIT for a small project because it is simple and more flexible. But if I want my project to always stay open and not be turned into closed software, then GPL would be better.

There is also a difference between “free as in free beer” and “free as in freedom”. Free beer means you get something for free, like no cost. But free as in freedom means you have control over it. In Git, it is more about freedom, because users can actually see the code, change it, and use it however they want within the license terms.

A3. The ethics of open source

Open source is not just about license, it’s more like a mindset of sharing things. But honestly I feel it’s not as simple as saying everything should be open source.

Like if I think about it, open source is really helpful because anyone can see the code and learn from it. It also makes software more transparent, like you know what is happening inside. But at the same time I don’t think all software can be open source. For example, companies that deal with sensitive data or security systems probably cannot make everything public. Also companies invest money, so it kind of makes sense why they keep some things private. So I feel both open and closed source have their own place.

About companies making profit using open source, this one is a bit confusing to judge. On one side it feels unfair because the base work is done by community for free. But on the other side, companies like Red Hat or Google don’t just use it, they also improve it and contribute back. So maybe it’s okay if they are also giving something in return. If a company just uses open source and never contributes, then it feels more wrong.

“Standing on the shoulders of giants” basically means building on top of existing work. In software this happens everywhere. Like no one is writing everything from scratch now. We use libraries, frameworks, etc. I think open source actually helps innovation because it saves time. Instead of doing basic things again, people can focus on new ideas. So I don’t think it reduces originality, it actually supports it.

So overall I think open source is really important, but it also depends on how people use it. It’s not automatically good or bad.

Part B — Linux Footprint (Unit 2)

I installed git on Ubuntu (VM). I already used git before for basic commands but never checked where it actually sits in Linux.

Installation:
Used apt:
sudo apt install git

It was straightforward. I think it also installed some extra packages but I didn’t check all of them. Didn’t try source install.

Directories:
git binary → /usr/bin/git

One thing I noticed (this was new for me) is that git doesn’t store everything in system folders. Instead, when we create a repo, it makes a .git folder inside the project directory itself. That folder contains all commits and history. I checked it once but didn’t fully understand all files.

Configs also exist (like global config), but I didn’t explore deeply.

User:
git runs as current user, not as root service. Like when I ran commands, it was just my user.

I think this is fine because git is not a server, it’s just a tool. Permissions still matter though when pushing or editing files.

Service:
No systemctl here (I first thought there will be one, but no). Git is not a background service.

Commands I used:
git init
git clone
git status

So it only runs when we execute something.

Updates:
Through apt:
sudo apt update
sudo apt upgrade

That's how it gets patches. I didn't manually track versions.

Overall:
Git feels different from other software because it's not running all the time. It's more like a utility you use when needed. The .git folder thing was probably the most interesting part for me.

Part C — The FOSS Ecosystem (Units 3 & 4)

I looked into Git and one thing I noticed is that it's not really independent. It depends on other tools and also supports many other systems.

Dependencies:
While installing Git (I used apt), I saw some extra packages getting installed. I didn't check all of them properly because there were too many names, but I think Git uses some system libraries and also works with SSH for remote repositories. I remember using SSH keys once, so I think that is connected.

What it has enabled:
Git is actually used by platforms like GitHub and GitLab. Earlier I thought Git and GitHub are same but they are not. Git is just the version control system, and GitHub is like a platform built on top of it. So Git basically made these platforms possible. There are probably many more tools like this.

Relation to web / LAMP:
Git is not directly part of LAMP (Linux, Apache, MySQL, PHP), but it is still used in web development. Like when people build websites or apps, they use Git to manage code. So it is kind of indirectly connected.

Community:
Git has a big community. I checked GitHub (obviously) and saw a lot of discussions and repositories. There are also issues where people report bugs. I didn't go into forums much, but it is clear that many developers are involved. I think the main maintainers handle core updates.

Overall, Git is more like a base tool that other things depend on rather than something isolated.

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Based on what I learned, I would recommend Git for real-world deployment. Even though it feels difficult in the beginning, especially with commands, it becomes more useful once we get used to it. I personally faced some confusion while using commands like commit and push, but after trying a few times it started making sense. It gives better control over version management, which is important in actual projects.
About contributing, I am not fully confident right now because I am still learning Git properly. But I think in the future, once I understand it better, I would like to try contributing. Since it is open source, anyone can contribute, which is actually a good thing. For now, I am just focusing on using it correctly in my own work, like when I used git push while working on my DSA problems.

Dimension	Open Source (Git)	Proprietary Alternative(Github Desktop)
Cost	Git is free. Like fully free, I didn't have to pay anything while installing or using it. That makes it easy for students also.	GitHub Desktop is also free to download, but since it is part of GitHub/Microsoft, I think for some advanced or company usage there can be cost involved.
Security (who can audit the code?)	Anyone can see the code of Git because it is open source. So if there is a bug, people can find it and fix it. But obviously it depends on people actually checking it.	In this, we cannot see the code. So we just trust the company that everything is secure. If something goes wrong, we have to wait for update.
Support and reliability	Support is mostly from internet. I had to search a lot when I got errors while using git commands. Sometimes answers are helpful, sometimes confusing also.	This is easier because everything is UI based. Less chances of mistakes. Also feels more stable for basic use.
Freeom to modify	Git can be changed or customized since it is open. I didn't try modifying it, but I know it is possible.	No modification. We just use whatever features are given.
Community vs corporate control	Git is handled by open source contributors. No single company is fully controlling it.	This is controlled by Microsoft, so whatever changes happen are decided by them.
Your overall verdict	Git is better if someone wants to learn properly and use in long term. It is difficult at first but useful later.	GitHub Desktop is good for starting, but not enough for deeper understanding or bigger projects.

4. Shell Script Tasks

You will write five shell scripts. Each script must be submitted as a plain .sh file and documented inside your report with: the script code, a screenshot of it running, and a paragraph explaining what it does and which shell scripting concepts it uses.

Important	Scripts must run on a real Linux system. They must include comments explaining each section. Uncommented scripts will lose marks.

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

```
MINGW64:/c/Users/aashw/OneDrive/Desktop
  GNU nano 8.7                                                    system_identity.sh
#!/bin/bash
# Script 1: System Identity Report
# By: Aashwit Suhane

# Variables
STUDENT_NAME="Aashwit Suhane"

# System info
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME="N/A (Git Bash on Windows)"   # Windows Git Bash does not have 'uptime'
DISTRO="Git Bash on Windows"
DATE=$(date)

# Display
echo "Open Source Audit - $STUDENT_NAME"
echo
echo "Kernel : $KERNEL"
echo "User   : $USER_NAME"
echo "Uptime : $UPTIME"
echo "Distro : $DISTRO"
echo "Date   : $DATE"
echo
echo "License : Git Bash uses MSYS2 license"
echo "System check completed"
```

```
aashw@Aashwit MINGW64 ~/OneDrive/Desktop
$ ./system_identity.sh
Open Source Audit - Aashwit Suhane

Kernel : 3.6.6-1cdd4371.x86_64
User   : aashw
Uptime : N/A (Git Bash on Windows)
Distro : Git Bash on Windows
Date   : Tue Mar 31 18:39:55 IST 2026

License : Git Bash uses MSYS2 license
System check completed
```

Script 2 — FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

```
MINGW64:/c/Users/aashw/OneDrive/Desktop
  GNU nano 8.7                                                    foss_package.sh
#!/bin/bash
# Script 2: FOSS Package Inspector
# By: Aashwit Suhane

# Variables
PACKAGE="git"

# Check installation
STATUS=$(git --version 2>/dev/null)

echo "Checking package: $PACKAGE"
echo

if [ -n "$STATUS" ]
then
    echo "Status  : Installed"
    echo "Version : $STATUS"
else
    echo "Status  : Not Installed"
fi

echo

# Description using case
case $PACKAGE in
    git)
        echo "Description : Git is used for version control and tracking code changes."
        ;;
    *)
        echo "Description : No info available"
        ;;
esac
```

```
aashw@Aashwit MINGW64 ~/OneDrive/Desktop
$ ./foss_package.sh
Checking package: git

Status  : Installed
Version : git version 2.53.0.windows.2

Description : Git is used for version control and tracking code changes.
```

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

```
MINGW64:/c/Users/aashw/OneDrive/Desktop
  GNU nano 8.7                                                    disk_permission.sh
#!/bin/bash
# Script 3: Disk and Permission Auditor
# By: Aashwit Suhane

# Variables
DIRECTORIES="/c /c/Users /c/Windows /c/Temp"

echo "Disk and Permission Report"
echo "--------------------------"

for DIR in $DIRECTORIES
do
    echo
    echo "Directory : $DIR"

    # Disk usage (Git Bash uses 'df -h', but paths need /c/... format)
    USAGE=$(df -h "$DIR" 2>/dev/null | tail -1 | awk '{print $5}')
    if [ -z "$USAGE" ]; then USAGE="N/A"; fi
    echo "Usage     : $USAGE"

    # Permissions
    PERM=$(ls -ld "$DIR" 2>/dev/null | awk '{print $1}')
    OWNER=$(ls -ld "$DIR" 2>/dev/null | awk '{print $3}')

    if [ -z "$PERM" ]; then PERM="N/A"; fi
    if [ -z "$OWNER" ]; then OWNER="N/A"; fi

    echo "Owner     : $OWNER"
    echo "Perms     : $PERM"
done
```

```
aashw@Aashwit MINGW64 ~/OneDrive/Desktop
$ ./disk_permission.sh
Disk and Permission Report
--------------------------

Directory : /c
Usage     : 44%
Owner     : aashw
Perms     : drwxr-xr-x

Directory : /c/Users
Usage     : 44%
Owner     : aashw
Perms     : drwxr-xr-x

Directory : /c/Windows
Usage     : 44%
Owner     : aashw
Perms     : drwxr-xr-x

Directory : /c/Temp
Usage     : 44%
Owner     : aashw
Perms     : drwxr-xr-x
```

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

```
MINGW64:/c/Users/aashw/OneDrive/Desktop
  GNU nano 8.7                                                    log_file.sh
#!/bin/bash
# Script 4: Log File Analyzer
# By: Aashwit Suhane

# Input file
FILE=$1
COUNT=0

echo "Analyzing file: $FILE"
echo

if [ ! -f "$FILE" ]; then
    echo "Error: File does not exist!"
    exit 1
fi

while read LINE
do
    if echo "$LINE" | grep -q "ERROR"
    then
        COUNT=$((COUNT+1))
    fi
done < "$FILE"

echo "Total ERROR lines : $COUNT"
```

```
aashw@Aashwit MINGW64 ~/OneDrive/Desktop
$ ./log_file.sh
No log files found in the current directory!
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

```
MINGW64:/c/Users/aashw/OneDrive/Desktop
  GNU nano 8.7                                                    manifestor_generator.sh
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# By: Aashwit Suhane

echo "Answer the following:"
echo

# Input
read -p "Your name: " NAME
read -p "Why do you like open source? " REASON
read -p "Your favorite tool: " TOOL

DATE=$(date)
FILE="manifesto.txt"

echo "Generating manifesto..."
echo

echo "On $DATE, $NAME shared their thoughts on open source." > "$FILE"
echo "$NAME believes that open source is important because $REASON." >> "$FILE"
echo "One tool they like working with is $TOOL." >> "$FILE"

echo "Saved to $FILE"

# Alias example (concept)
# alias runmanifesto="bash manifesto.sh"
```

```
aashw@Aashwit MINGW64 ~/OneDrive/Desktop
$ ./manifestor_generator.sh
Answer the following:

Your name: Aashwit Suhane
Why do you like open source? cost savings, transparency, and the ability to customize solutions
Your favorite tool: Git
Generating manifesto...

Saved to manifesto.txt
```

5. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths — but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	*Choice confirmed*
2	**Philosophy**	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	*Draft A1*
3	**License & Ethics**	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	*Draft A2 + A3*
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	*Draft Part B + screenshots*
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	*Scripts 1–2 working*
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	*Scripts 3–4 working*
7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	*Draft Part C*
8	**Comparison & Script 5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	*Draft Part D + Script 5*
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	*Complete draft*
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	*Final submission*

6. Evaluation Rubric

Total marks: 100. The report carries 60 marks and the shell scripts carry 40 marks. There is no viva unless the evaluator suspects the work is not original — in which case any student may be called for a brief oral explanation.

Report (60 marks)

#	Component	Full marks	Passing (50%)	Notes
1	Origin story — depth, original research, storytelling	**12**	6	*Part A1*
2	License analysis — accuracy and understanding of freedoms	**12**	6	*Part A2*
3	Ethical reflection — quality of argument and original thinking	**10**	5	*Part A3*
4	Linux footprint — technical accuracy and screenshots	**10**	5	*Part B*
5	Ecosystem and LAMP connection	**8**	4	*Part C*
6	Open source vs proprietary comparison	**8**	4	*Part D*

Shell Scripts (40 marks — 8 marks each)

Each script is marked on three criteria:

Criterion	Marks	What we look for
Correctness — does it run without errors?	**4**	*Script executes on Linux and produces expected output*
Concepts — does it use the right shell constructs?	**2**	*Loops, if-then, case, variables used appropriately for the task*
Comments and documentation	**2**	*Every non-obvious line has a comment; script purpose is clear*

7. Reference Resources

Use these as starting points, not as sources to copy from. The report must be in your own words.

On open source philosophy

- GNU Project — 'The Free Software Definition': gnu.org/philosophy/free-sw.html
- Open Source Initiative — 'The Open Source Definition': opensource.org/osd
- Eric S. Raymond — 'The Cathedral and the Bazaar': catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts — freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab — the LICENSE file is always there

A closing thought

Every tool you will use in your career — the editor, the compiler, the server, the database — was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice, and what it means for the world, is not a footnote to your technical education. It is the foundation of it.

8. Submission Requirements

All submissions are made through the VITyarthi portal. You must submit three things: a public GitHub repository link, a README.md inside the repository, and a project report PDF. All three are compulsory. Incomplete submissions will not be evaluated.

What to submit on the VITyarthi portal

Item	Details	Compulsory?
GitHub repo link	The repository must be public. It must contain all five .sh script files and a README.md. Name the repo: oss-audit-[rollnumber]	**Yes**
README.md	Must be inside the GitHub repository. Must include: student name and roll number, chosen software, description of each script, step-by-step instructions to run each script on Linux, and any dependencies required.	**Yes**

Project report PDF	Upload directly on the VITyarthi portal. Must follow the structure in Section 3. Must be 12 to 16 pages. Must be your own writing.	**Yes**

On the VITyarthi portal submission form, paste the GitHub repository URL, confirm the README is present in the repository, and upload the project report PDF. All three must be submitted together in a single submission — partial submissions will not be accepted.

Evaluation pipeline

Every submission goes through an automated evaluation pipeline before it reaches an evaluator. Be aware of the following checks that run on every submitted report and script set.

Check	What it detects	Consequence
Plagiarism detection	Report text copied from websites, Wikipedia, other students' submissions, or any published source without attribution. Cross-submission similarity between enrolled students is also flagged.	Zero on the full report. Referred to disciplinary committee if similarity exceeds 40%.
AI generation detection	Report sections generated by AI tools such as ChatGPT, Gemini, Claude, or similar. The pipeline checks for linguistic patterns, sentence uniformity, and structural signatures common to AI-generated text.	Flagged submissions are called for a mandatory viva. If the student cannot explain the content, zero marks are awarded.

A note on AI tools: you may use AI to understand concepts, debug scripts, or get explanations. You may not use AI to write your report. The distinction is the same as using a calculator versus asking someone else to sit your exam. The report must reflect your thinking, your words, and your conclusions.